

MANIPAL INSTITUTE OF TECHNOLOGY

(A constituent unit of Manipal Academy of Higher Education)

School of Computer Engineering

MINI PROJECT REPORT

Parallel Computer Architecture and Programming
(CSE-3129)

Heterogeneous Parallelization of an Agent-Based Limit Order Book Simulator Using CUDA, OpenMP, and MPI

Submitted by:

Name	Registration Number	Roll Number
Abhisaar Gupta	240962059	10
Devesh Khaitan	240962031	3
Sohan Kar	240962358	23
Pratham Dobhal	240962422	29

Source code:
`https://github.com/abhisaarguptaa/Stock-market-simulator-using-PCAP`

October 2026

Heterogeneous Parallelization of an Agent-Based Limit Order Book Simulator Using CUDA, OpenMP, and MPI

Abhisaar Gupta, Devesh Khaitan, Sohan Kar, and Pratham Dobhal

School of Computer Engineering, Manipal Institute of Technology,
Manipal Academy of Higher Education, Manipal, India

Abstract. Agent-based simulators of limit order book (LOB) markets reproduce how prices form from the orders of many independent traders, but mainstream simulators run as single-threaded event loops and scale poorly with the number of agents and instruments. The workload is not uniformly parallel: agents decide independently, which suits a GPU, while price–time priority forces the orders of one instrument to be matched one at a time. We implement a deterministic simulator with a three-tier heterogeneous mapping (agent decisions in CUDA, matching parallelised across instruments with OpenMP, and instruments distributed across MPI ranks) and measure where its performance limits lie. Every back end produces a bit-identical trade log for a given seed, verified on 12 configurations covering 1.29 million trades. On a Tesla T4 the decision kernel is about 85× faster than two CPU threads, yet whole-tick speedup is bounded at 1.11–1.41× because matching takes 92–96% of each tick. CUDA overtakes the multicore baseline at 4,000–16,000 agents. On a 16-thread laptop, matching speeds up only across instruments (0.90× with one instrument, 3.97× with 128); strong scaling peaks at 1.99× on 8 threads with an effective serial fraction of 0.49. The simulated returns show heavy tails and volatility clustering. The results show that order matching under price–time priority, and not the agent computation, is the scaling bottleneck on every back end.

Keywords: Limit order book · Agent-based simulation · Heterogeneous computing · CUDA · OpenMP · MPI · Amdahl's law · Reproducibility

1 Introduction

A financial exchange is, at its core, a large concurrent system. Thousands of independent participants form intentions at the same time, but their orders must be serialised into a single, strictly ordered book per traded instrument before any trade can occur. Agent-based models (ABMs) of limit order book (LOB) markets reproduce this structure in software and are widely used to study price formation, liquidity, and the statistical regularities of returns [1,2]. Their main practical limitation is computational: fidelity improves with the number of simulated agents and instruments, yet mainstream market simulators execute as single-threaded discrete-event loops [3] and therefore scale poorly with either.

This workload is interesting for a parallel computer architecture study because it is not uniformly parallel. The agent decision phase is massively data-parallel and maps naturally onto a GPU, since every agent independently reads a shared view of the book and emits at most one order. The matching phase is the opposite: price–time priority imposes a total order on events within an instrument, so matching for a single instrument resists parallelisation almost entirely and can only be distributed across instruments. The simulation therefore contains an irreducible serial core between two parallel regions, and the achievable speedup is governed by the interaction between them rather than by the efficiency of any single kernel.

This project implements such a simulator with a three-tier heterogeneous mapping: agent decisions in CUDA, per-instrument matching parallelised across instruments with OpenMP, and instrument sets distributed across processes with MPI. We treat the resulting performance envelope as the object of study. The main contributions are:

- A deterministic LOB simulator whose serial, OpenMP, CUDA and MPI back ends produce bit-identical trade logs for a given seed, verified by an order-sensitive hash of every trade.
- Measurements of the CPU–GPU boundary cost, the effect of warp divergence and copy-back compaction, and the agent count at which heterogeneous execution overtakes a multicore baseline.
- A strong, weak and per-instrument scaling study with Amdahl, Gustafson, Karp–Flatt and roofline analysis, showing that order matching is the bottleneck on every back end.
- An open-source implementation with one-command setup and demo scripts, so every result in this report can be reproduced.

The rest of the report is organised as follows. Section 2 reviews related work, Sections 3 and 4 state the problem and objectives, Sections 5–7 describe the method, architecture and algorithms, Sections 8–10 present the experiments, results and analysis, Section 11 compares the work with existing methods, and Sections 12–13 conclude.

2 Literature Review

2.1 Foundations

Gode and Sunder showed that agents with no intelligence beyond a budget constraint already produce near-efficient allocations in a double auction, so much of market behaviour comes from the trading mechanism itself [4]. Chiarella and Iori added fundamentalist, chartist and noise traders interacting through an explicit order book [5]; Preis et al. showed that a simple multi-agent order book with price–time priority reproduces fat tails once order placement depth depends on the trend [2]; and Farmer et al. found that zero-intelligence order flow explains much of the observed spread and price impact [6]. Cont's stylised facts

(heavy tails, volatility clustering and no linear autocorrelation) are the standard checklist for validating such models [1]. On the computing side, Lysenko and D'Souza built the first largely GPU-resident ABM framework [7], Fujimoto surveyed parallel discrete-event simulation [8], and Amdahl [9], Gustafson [10], Karp and Flatt [11] and Williams et al. [12] give the performance models used in this report.

2.2 Recent agent-based market simulators

ABIDES-Gym [13] (ACM ICAIF 2021) wraps ABIDES [3], a message-driven discrete-event simulator with a NASDAQ-like exchange and price–time priority, as an OpenAI Gym environment for reinforcement learning. It offers high fidelity and realistic latency, but its kernel is a single-threaded event loop and the paper reports no run times; a later study cites ABIDES at about 29,000 messages per second [14].

JAX-LOB [15] (ACM ICAIF 2023) is the first GPU-accelerated LOB simulator. Each book is a fixed-capacity array, and matching runs inside a JAX loop. The authors note that processing messages within one book is inherently serial, so all parallelism comes from running thousands of independent books with `vmap`: with 1000 books, a message costs about 2.6 μs per book on an RTX 2080 Ti, and RL training is about 7× faster. Its limits are pre-allocated capacity, a cost set by the slowest branch, and replayed order flow that does not react to the agent. It does not measure the end-to-end serial fraction or combine GPU agents with CPU matching.

PyMarketSim [14] (ACM ICAIF 2024) is a Python discrete-time simulator whose tick (observe, act, clear, advance) matches ours. Its order book takes 1.9–4.7 μs per operation from 10^3 to 10^6 orders, about 300,000 book operations or 15,000 agent actions per second, but it is serial, simulates one asset and uses 25 agents in its experiments.

2.3 GPU and high-performance ABM frameworks

FLAME GPU 2 [16] (Software: Practice and Experience 2023) compiles agent functions to CUDA at run time, stores agents as structures of arrays, and passes messages through spatial, array or bucket structures. It runs up to 200 million agents on a V100 and 1M agents in about 3 ms per step. Its performance depends on spatial or keyed locality; agents competing for one resource need repeated bidding sub-models, atomics are non-deterministic, and one model is limited to one GPU.

Agents.jl [17] (SIMULATION 2024, online 2022) is a Julia ABM library that runs single models about 2–125× faster than MASON, NetLogo and Mesa in the authors' benchmarks. It does not parallelise within a model, because agents mutate shared state, and has no GPU support: the same shared-state conflict that an order book creates.

2.4 Matching engines and hardware-accelerated trading

CoinTossX [18] (SoftwareX 2022) is an open-source Java matching engine modelled on the Johannesburg Stock Exchange, with price–time priority and messaging over UDP. It reaches 106–248 ns 90th-percentile matching latency on a 16-core server, but throughput falls from 2,287 to 621–989 orders per second as client–stock pairs are added, and it is engineered for latency rather than for parallel scaling.

LightTrader [19] (IEEE HPCA 2023) is a heterogeneous high-frequency trading system: an FPGA parses market data and keeps the top of a local order book, while custom deep-learning accelerators run price-prediction models, with a scheduler that picks batch size and voltage and frequency under a power budget. It is 13.9× faster than a V100-based system for AI processing, but scaling saturates as accelerators share the power budget. It accelerates the *decision* stage for one trader, the analogue of our GPU phase.

2.5 Distributed ABM and reproducible parallel random numbers

Repast4Py [20] (Winter Simulation Conference 2022) distributes Python ABMs with MPI, partitioning space across ranks with ghost regions. A 3.6M-agent random walk speeds up 4.39× from 36 to 288 ranks (efficiency 0.94 to 0.55), while a network model reaches only 1.75× (efficiency 0.22): communication dominates at higher rank counts. Whether results are identical across rank counts is not addressed.

SIMCoV-GPU [21] (ACM HPDC 2024) ports a spatial infection ABM to multiple GPUs with CUDA and UPC++, reaching 11.9× over 512 CPU cores with 16 GPUs. The GPU wins only on large enough problems (at 64 GPUs on a small grid it is 0.85×), and because CPU and GPU random number generators differ, results agree only statistically (within about 1%), not bit for bit.

OpenRAND [22] (SoftwareX 2024) provides counter-based generators (Philox, Threefry and others) keyed by a logical entity rather than a thread, so streams give the same output on any thread count or device; it is 1.8× faster than cuRAND in a particle benchmark. It supplies reproducible random numbers but not reproducibility of a whole simulation, which also needs deterministic arithmetic and event ordering.

2.6 Summary, research gaps and motivation

Table 1 summarises the reviewed work. Four gaps motivate this project:

1. **Market simulators are built for fidelity, not parallel characterisation.** ABIDES-Gym, PyMarketSim and CoinTossX run essentially serially; JAX-LOB parallelises only across independent books. None splits the tick into a parallel decision phase and a serial matching phase and relates the measured speedup to Amdahl's law.

2. **The serial matching core is acknowledged but not measured.** No study reports what share of tick time matching takes as agents grow, or the gap between kernel and end-to-end speedup.
3. **GPU and distributed ABM frameworks rely on locality.** FLAME GPU 2, Repast4Py and SIMCoV-GPU exploit spatial neighbourhoods and halos; Agents.jl avoids in-model parallelism. An order book, where every agent writes into one strictly ordered structure per instrument, fits none of them.
4. **Bit-identical heterogeneous results are rare.** SIMCoV-GPU accepts statistical agreement and FLAME GPU 2 has non-deterministic atomics; OpenRAND makes only the random numbers reproducible. No reviewed work shows a bit-identical event log across CUDA, OpenMP and MPI.

We therefore build a simulator that partitions by instrument, keeps the GPU for decisions only, produces identical trades on every back end, and measures where its speedup stops and why.

3 Problem Statement

Given a population of N heterogeneous trading agents and M instruments, each with its own limit order book, simulate T discrete ticks in which (i) every agent observes a snapshot of its instrument's book and may submit, cancel or modify one order, (ii) each instrument's orders are matched under price–time priority, and (iii) a market-wide index is published for the next tick. The problem is to map this pipeline onto heterogeneous hardware (a GPU, a multicore CPU and multiple processes) such that:

1. the parallel versions produce *exactly* the same trades as a serial reference for the same random seed, so that correctness can be checked and experiments repeated;
2. the speedup over the serial reference is measured and explained, including the serial fraction imposed by price–time priority, the cost of moving data across the CPU–GPU boundary, and the cost of synchronising MPI ranks at every tick; and
3. the agent and instrument counts at which the heterogeneous mapping beats an optimised multicore CPU implementation are identified.

The simulated market must also remain realistic, reproducing the standard stylised facts of financial returns [1].

4 Objectives

The eight objectives from the project synopsis, and where each is addressed, are:

1. **Serial matching engine.** Implement a correct serial LOB engine supporting add, cancel, modify and market orders under price–time priority, with a unit test suite as the correctness reference (Sections 5, 7, 9.1).

Table 1. Comparison of recent related work (2021–2024).

Work	Approach / platform	Advantages	Limitations	Key result
ABIDES-Gym [13]	discrete-event kernel, Python, serial	high fidelity, latency model, RL interface	single-threaded; no run times reported	–
JAX-LOB [15]	JAX on GPU, `vmap` over books	thousands of books in parallel; RL on GPU	serial within a book; fixed capacity	2.6 μs/msg per book; 7× RL
PyMarketSim [14]	Python discrete-time tick	simple, RL and game-theory tools	serial; one asset; 25 agents	300k book ops/s
FLAME GPU 2 [16]	CUDA, SoA, spatial messages	200M agents on one GPU	needs locality; non-det. atomics; 1 GPU per model	1M agents in 3 ms/step
Agents.jl [17]	Julia, CPU	concise and fast single models	no in-model parallelism; no GPU	2–125× vs other ABM tools
CoinTossX [18]	Java engine, UDP	sub-μs matching latency	throughput falls with instruments	106–248 ns latency
LightTrader [19]	FPGA + DL accelerators	fast decision stage, energy-aware	power-limited scaling; one trader	13.9× vs V100 system
Repast4Py [20]	Python + MPI, spatial partitions	HPC scale from Python	communication-bound at high rank counts	4.39×, 36→288 ranks
SIMCoV-GPU [21]	CUDA + UPC++, multi-GPU	11.9× over 512 cores	GPU wins only at large sizes; not bitwise reproducible	11.9× (16 GPUs)
OpenRAND [22]	counter-based RNG library	reproducible on any back end	RNG only, not whole simulation	1.8× vs cuRAND

2. **Agents and serial baseline.** Implement noise, momentum, mean-reverting and market-making agents and a fully serial simulator as the speedup baseline (Section 5).
3. **CUDA agent phase.** Port agent decisions to CUDA with agent-type sorting to reduce warp divergence, coalesced access, and counter-based random streams for determinism (Sections 7, 9.2).
4. **OpenMP matching.** Parallelise matching across instruments and evaluate thread scaling, false sharing and memory contention (Section 9.3).
5. **MPI distribution.** Distribute instruments across ranks with a per-tick index reduction and quantify communication overhead (Section 9.3).
6. **Validation.** Compare trade logs exactly across back ends and check the stylised facts (Sections 9.1, 9.4).

7. **Scaling study.** Strong and weak scaling, an Amdahl fit of the serial fraction, roofline placement and Nsight metrics (Sections 9, 10).
8. **CPU vs GPU threshold.** Identify where the heterogeneous mapping overtakes the multicore CPU and explain why (Sections 9.2, 10).

5 Proposed Methodology

5.1 Barrier-synchronised tick

The simulator advances in discrete ticks. Following the conservative synchronisation family of parallel discrete-event simulation [8], every tick has three phases separated by barriers:

1. **Decide** (parallel over agents): each agent reads its instrument's snapshot (best bid, best ask, last price, trend) and the market index, and emits at most one order intent.
2. **Match** (parallel over instruments, serial inside each): the intents of each instrument are applied to its book in a deterministic arrival order, producing trades.
3. **Publish** (reduction over instruments): snapshots for the next tick are rebuilt and the market index (the sum of mid prices) is reduced, across ranks with `MPI_Allreduce`.

This trades some concurrency for determinism and auditability; the cost of the barrier is one of the quantities measured.

5.2 Order book

Each instrument has a fixed grid of 4096 integer price levels with one FIFO queue per level. Orders live in a node pool and are doubly linked into their level, with an id-to-node hash map, so add, cancel and modify are $O(1)$ except for moving the best-price pointer when a level empties. Limit orders match against the opposite side at the resting (maker) price and the remainder rests; market orders match and drop any unfilled remainder. A modify keeps time priority only for a quantity decrease at the same price. Resting orders expire after a time-to-live (100 ticks by default), which keeps the book bounded.

5.3 Agent population

Four strategies, after the heterogeneous populations of [5,2], are written once as `__host__ __device__` functions so that the CPU loops and the CUDA kernel run literally the same code (Table 2). Agents are stateless functions of (agent id, tick, seed, snapshot): they do not track inventory, which keeps the GPU phase free of per-tick feedback from the matching engine.

Table 2. Agent population (default mix).

Strategy	Share	Acts/tick	Behaviour
Noise	65%	10%	random side; mostly limit orders near the reference price
Momentum	15%	5%	market order in the direction of the instrument and index trend
Mean-reverting	10%	3%	limit order back toward the fundamental value
Market maker	10%	100%	re-quotes one side per tick, cancels its quote from two ticks earlier

5.4 Determinism by design

Parallel stochastic simulations are usually not repeatable, because random draws and floating-point reductions depend on thread scheduling. We make every back end produce the identical trade log:

- **Counter-based RNG:** Philox4x32-10 [23] maps (seed, stream, agent, tick) to random numbers with no shared state, so thread order cannot change any draw.
- **Integer-only decisions:** CPU and GPU floating point can round differently (fused multiply–add, math libraries), so decisions use integer arithmetic.
- **Deterministic arrival order:** intents are applied in agent-id order, rotated by a random offset per (instrument, tick) so that no agent is always first.
- **Predictable order ids:** an order's id is $(tick+1) \ll 32 \mid agent$, so a market maker on the GPU can cancel its earlier quote without feedback from the CPU.
- **Integer reductions:** the index is a sum of integers, so `MPI_Allreduce` gives the same result in any reduction order.
- **Order-independent compaction:** GPU output slots are reserved with `atomicAdd`; each entry carries its agent index and the host scatters by index.

Correctness is checked with an order-sensitive FNV-1a hash over every trade (tick, price, quantity, buyer, seller, maker and taker ids), combined across instruments and ranks.

5.5 Parallel mapping

- **CUDA (decide):** one thread per agent, structure-of-arrays layout for coalesced loads, agents sorted by strategy so that warps mostly take one branch, snapshots read through the read-only cache (`__ldg`), parameters in constant memory, and compaction of the copy back to only the agents that acted (about 20% per tick).
- **OpenMP (decide and match):** `parallel for schedule(static)` over agents for decisions; `parallel for schedule(dynamic,1)` over instruments for matching, with per-thread counters padded to 64-byte cache lines to avoid false sharing.

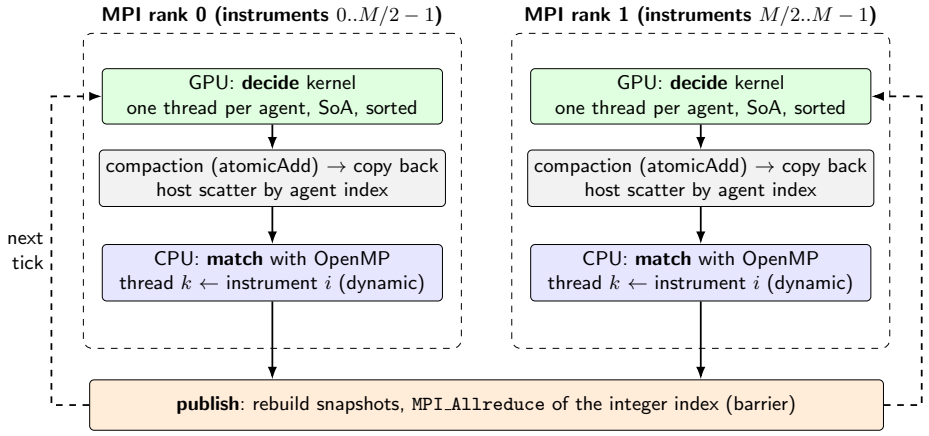


Fig. 1. System architecture: the three-tier heterogeneous mapping (two MPI ranks shown).

– **MPI (instruments):** each rank owns a contiguous block of instruments and the agents trading them; ranks meet once per tick in an integer `MPI_Allreduce`.

5.6 Measurement

The program times every phase of every tick (decide, host-to-device copy, kernel, device-to-host copy, match, publish, MPI communication) and reports totals; for MPI it reports the slowest rank. Kernel time uses CUDA events. Speedup is $S(p) = T_1/T_p$ against the serial back end, efficiency is $E(p) = S(p)/p$, the serial fraction f is fitted to Amdahl's law $S(p) = 1/(f + (1-f)/p)$ [9] by least squares, and the Karp–Flatt metric $e(p) = (1/S - 1/p)/(1 - 1/p)$ [11] is reported for each p. Weak scaling is interpreted with Gustafson's law [10] and the kernel is placed on a roofline [12] from Nsight Compute counters.

6 System Architecture

Fig. 1 shows how the three tiers map onto hardware and Fig. 2 shows the control flow of one simulation run. MPI splits the instruments into blocks, one per rank. Inside a rank, the decision phase runs on the GPU (or on all CPU cores with OpenMP), and the matching phase runs on the CPU with one instrument per OpenMP thread at a time. The ranks synchronise once per tick to reduce the market index. Table 3 lists the software modules.

7 Algorithms

Algorithm 1 is the top-level tick loop executed by every MPI rank. Algorithm 2 matches one incoming limit order under price–time priority; it is the serial core of the simulation. Algorithm 3 is the CUDA decision kernel.

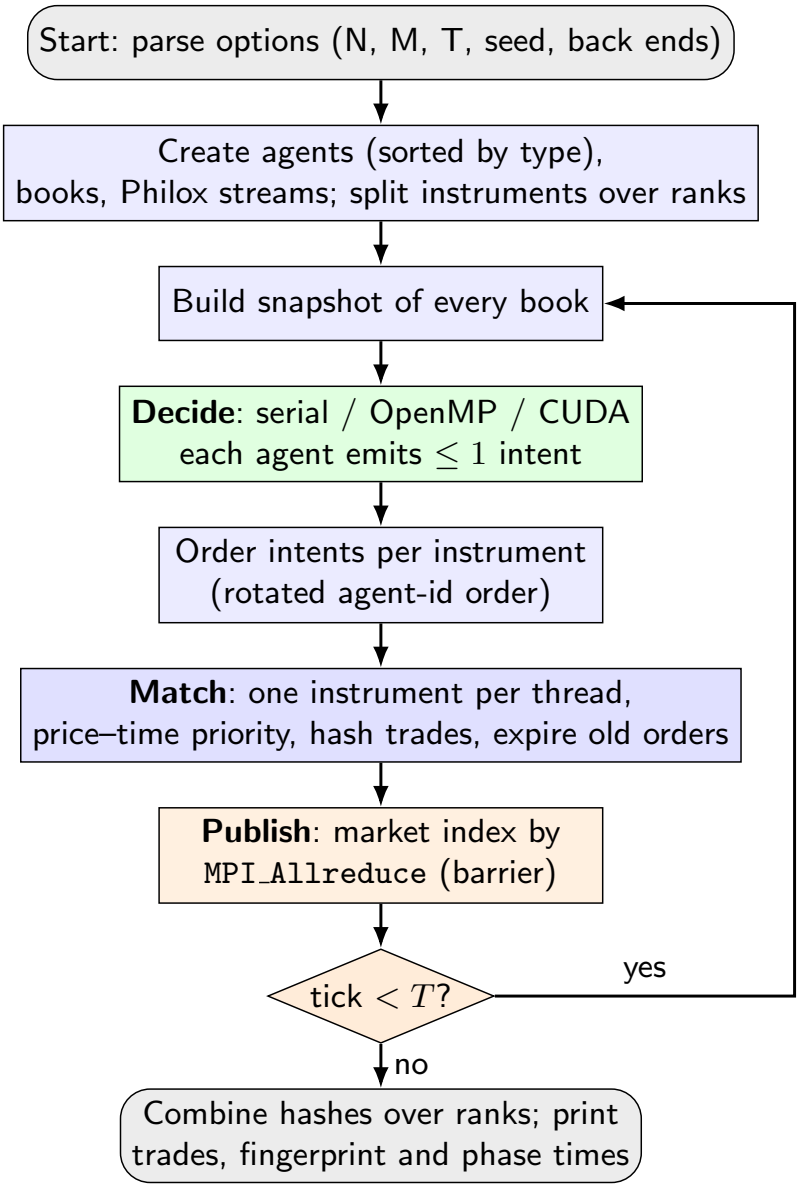


Fig. 2. Flowchart of one simulation run.

Complexity. Decisions cost $O(N)$ work per tick with $O(N/p)$ depth on p processors. Each order costs $O(1)$ to add or cancel plus $O(k)$ for k fills; with n_i orders on instrument i, matching costs $O(\sum_i n_i)$ work but its depth is $O(\max_i n_i)$, because the orders of one instrument are processed sequentially. With a single

Table 3. Software modules (C++17, CUDA, OpenMP, MPI).

File	Role
`order_book.hpp/.cpp`	serial matching engine, price–time priority
`agents.hpp`	four strategies, shared by CPU and GPU
`philox.hpp`	Philox4x32-10 counter-based RNG
`sim.hpp/.cpp`	tick loop, OpenMP matching, MPI reduction, trade hash
`decide_cuda.cu`	CUDA decision kernel and compaction
`main.cpp`	`lobsim` command-line program
`test_main.cpp`	unit tests (259,597 checks)
`scripts/`, `analysis/`	setup, demo, benchmarks, profiling, plots

Algorithm 1 Barrier-synchronised tick loop (per MPI rank)

Input: agents A_r and instruments I_r owned by rank r; ticks T; seed s

for $t \leftarrow 0$ **to** $T-1$ **do**
  $snap[i] \leftarrow \textsc{Snapshot}(book[i])$ **for all** $i \in I_r$
  // Phase 1: decide (CUDA kernel or OpenMP parallel for)
  for all $a \in A_r$ **in parallel do**
    $intent[a] \leftarrow \textsc{Decide}(a, t, s, snap[inst(a)], index)$
  end for
  // Phase 2: match, one instrument per thread
  for all $i \in I_r$ **in parallel** (`schedule(dynamic,1)`) **do**
    $o \leftarrow \textsc{Rotate}(\text{agents of } i, \textsc{Philox}(s, i, t))$
    for all a **in order** o with $intent[a] \neq \emptyset$ **do**
      $trades \leftarrow \textsc{Apply}(book[i], intent[a])$ ▷ Alg. 2
      $h[i] \leftarrow \text{FNV1A}(h[i], trades)$
    end for
    $\textsc{ExpireOrders}(book[i], t)$
  end for
  // Phase 3: publish
  $local \leftarrow \sum_{i \in I_r} mid(book[i])$ ▷ integer
  $index \leftarrow \text{MPI_Allreduce}(local, \textsc{Sum})$ ▷ barrier
end for
return $\text{MPI_Allreduce}(h, \textsc{BXor})$ ▷ per-instrument hashes

instrument the depth equals the work, which is the source of the serial fraction measured in Section 10.

8 Experimental Setup

8.1 Platforms

No single machine available to the team had both a GPU and many CPU cores, so experiments ran on two platforms (Table 4): GPU experiments on a Google Colab Tesla T4 (which has only 2 CPU threads), and CPU scaling experiments on a 16-thread laptop. A 4-core Linux machine was used for the live demo run and its screenshot.

Algorithm 2 Match a limit buy order (price–time priority)

Input: book with ask levels $L[p]$ (FIFO queues), best ask p_a; order (id, p, q)

while $q > 0$ **and** $p_a \le p$ **do**
  $m \leftarrow \textsc{Front}(L[p_a])$ ▷ oldest order at best price
  $x \leftarrow \min(q, m.q)$
  $\textsc{EmitTrade}(p_a, x, \text{maker} = m, \text{taker} = id)$
  $q \leftarrow q - x$; $m.q \leftarrow m.q - x$
  if $m.q = 0$ **then**
    remove m from $L[p_a]$ and from the id map
    if $L[p_a]$ is empty **then**
      $p_a \leftarrow$ next non-empty ask level
    end if
  end if
end while
if $q > 0$ **then**
  append (id, p, q) to the back of bid level p; update best bid
end if

Algorithm 3 CUDA decision kernel with compaction

Input: SoA arrays $type[], inst[]$ (sorted by type); snapshots in global memory; params in constant memory

$a \leftarrow blockIdx.x \cdot blockDim.x + threadIdx.x$
if $a \ge N$ **then return**
end if
$sn \leftarrow$ `__ldg`$(\&snap[inst[a]])$ ▷ read-only cache
$(r_0..r_3) \leftarrow \textsc{Philox4x32}(s, a, t)$ ▷ no shared RNG state
$o \leftarrow \textsc{Decide}_{type[a]}(sn, index, r)$ ▷ integer only; warps mostly uniform
if $o \neq \emptyset$ **then**
  $k \leftarrow$ `atomicAdd`$(\&count, 1)$
  $out[k] \leftarrow (a, o)$ ▷ slot order varies; agent index makes it irrelevant
end if
// Host: copy back $out[0..count - 1]$ *only, then* $intent[out[k].a] \leftarrow out[k].o$

8.2 Workloads and parameters

Unless stated otherwise, runs use the default agent mix (Table 2), 4096 price levels per instrument, an order lifetime of 100 ticks, and seed 42. The experiment sets were:

- **Determinism:** 50,000 agents, 16 instruments, 500 ticks, seed 1234, on 12 back-end configurations (Colab) and on serial, OpenMP 1–16 threads and MPI 1/2/4 ranks with 400,000 agents and 64 instruments (laptop).
- **CPU vs GPU crossover:** 1k to 2M agents, 128 instruments, 200 ticks.
- **Kernel experiments:** 1M agents, 128 instruments; block size 32–1024; type sorting and compaction on/off; Nsight Compute on one tick.
- **Agents at fixed instruments (Gustafson):** 32 instruments, 32k to 2M agents.

Table 4. Experimental platforms and software.

	GPU platform	CPU platform
Machine	Google Colab VM	laptop, Windows 11 + WSL2
CPU	2 vCPU (Intel Xeon)	16 threads (Intel, Iris Xe graphics)
GPU	NVIDIA Tesla T4 (Turing, sm_75), 2560 cores, 16 GB, 320 GB/s	none
OS	Ubuntu 24.04	Ubuntu 24.04 (WSL2)
Compiler	g++ 13.3, nvcc 13.0, `-O3 -march=native`	g++, `-O3 -march=native`
MPI	Open MPI 4.1.6	Open MPI
Profilers	Nsight Systems / Compute 2025.3	built-in phase timers
Used for	objectives 3, 6, 8; determinism	objectives 4, 5, 7; determinism

- **Instruments sweep:** 200,000 agents, 16 threads, 1 to 128 instruments, serial vs OpenMP matching.
- **Strong scaling:** 400,000 agents, 64 instruments, 300 ticks, 1 to 16 threads.
- **Weak scaling:** 50,000 agents and 8 instruments per thread, 300 ticks.
- **MPI:** 400,000 agents, 64 instruments, 1, 2 and 4 ranks (16 threads in total).
- **False sharing:** padded vs unpadded per-thread counters, 200,000 agents, 256 instruments, 16 threads.
- **Stylised facts:** 64,000 agents, 16 instruments, 20,000 ticks.

8.3 Methodology

Each configuration ran 3 times (Colab, first laptop run) or 5 times (laptop strong and weak scaling), and we report the median. For strong and weak scaling the configurations were *interleaved* (one run of each configuration per round), with the laptop plugged in and set to its best-performance power mode; a first, non-interleaved run was discarded because the serial and 1-thread OpenMP times differed by 60% under identical work. All raw CSV files are in the repository's `results/` folder, and every experiment is reproducible with `scripts/bench.sh`.

9 Results

9.1 Correctness and execution

All 259,597 unit-test checks pass, including a randomised test that compares the order book against a simple reference implementation and tests that compare each parallel back end against the serial one (Fig. 3). On the T4, all 12 configurations (serial; OpenMP with 1, 2, 4 and 8 threads; CUDA with and without sorting and compaction; MPI with 2 and 4 ranks using OpenMP and CUDA) produced the identical trade log: 1,289,848 trades with fingerprint `2b39a9b633ea375b` (Fig. 4). On the laptop, serial, OpenMP with 1 to 16 threads and MPI with 1, 2 and 4 ranks all gave fingerprint `cd3e16901eff5bb0` for 5,013,234 trades. Fig. 5

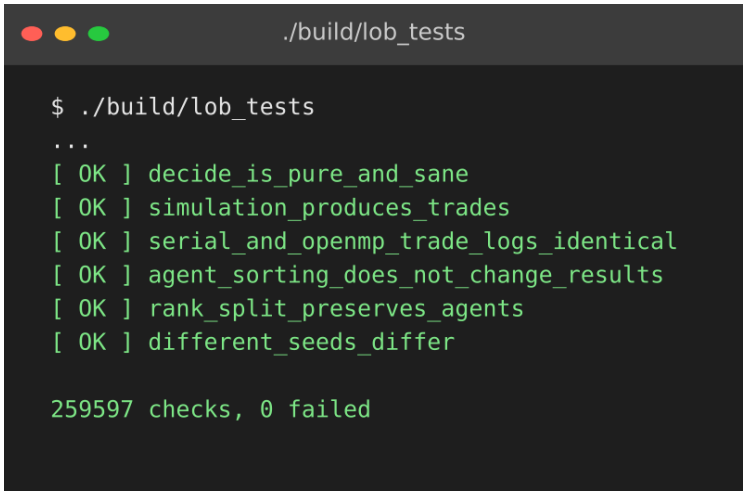


Fig. 3. Execution: unit test run (end of output).

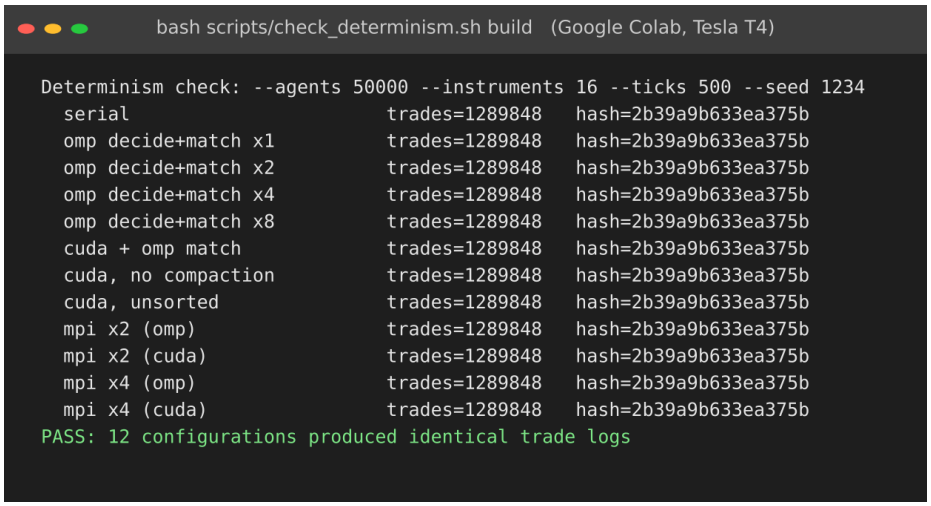


Fig. 4. Execution on the Tesla T4: determinism check across 12 back-end configurations (output of `scripts/check_determinism.sh`).

shows the live demonstration script: the input is the command line (agents, instruments, ticks, seed) and the output is the number of trades, the fingerprint and the time per back end. Fig. 6 shows the GPU environment check.

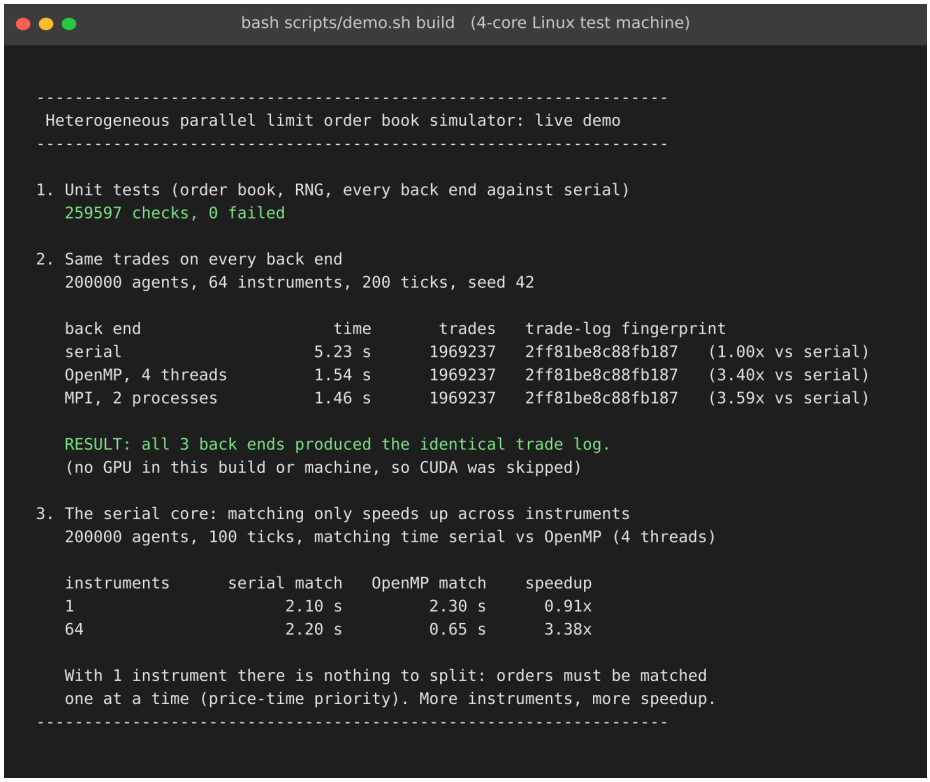


Fig. 5. Execution: the live demo (`bash scripts/demo.sh build`) on a 4-core Linux machine with 200,000 agents, 64 instruments and 200 ticks. Serial, OpenMP and MPI produce the same 1,969,237 trades and fingerprint; the last table shows that matching speeds up with 64 instruments but not with one.

9.2 GPU results (Tesla T4)

Table 5 breaks one tick down at 1M agents and 128 instruments. The CUDA kernel takes 0.16 ms against 13.7 ms for two CPU threads, about 85× faster, but the decision phase as a whole is only 3.6× faster: copying results back and scattering them into the agent array on the host cost more than the kernel. Matching is unchanged at 84 ms, so the whole tick improves only from 97.4 to 87.9 ms.

Fig. 7 and Table 6 show how CUDA compares with the OpenMP baseline as the number of agents grows. Below about 4,000 agents CUDA is slower: copies and kernel launches are a fixed cost, and copies alone are 30% of the tick at 1,000 agents. CUDA overtakes OpenMP between 4,000 and 16,000 agents and peaks at 1.41×. Beyond that the gain shrinks again, because matching grows with the number of orders and swallows the GPU's saving.

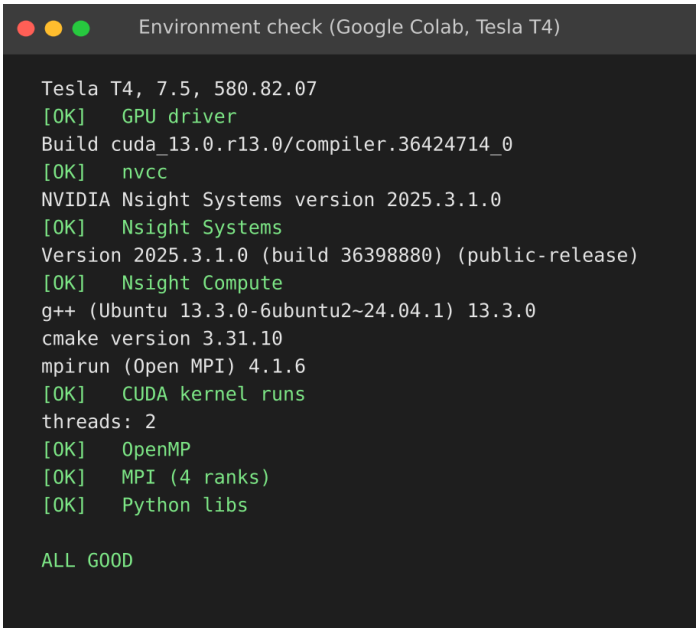


Fig. 6. Execution: Colab GPU environment check (CUDA 13, Nsight, Open MPI, OpenMP).

Table 5. Time per tick by phase, 1M agents, 128 instruments (T4).

Phase	OpenMP (2 thr.)	CUDA
Agent decisions	13.7 ms	3.8 ms
kernel	–	0.16 ms
copy back (compacted)	–	0.46 ms
host scatter	–	≈3.2 ms
Matching (CPU, OpenMP)	83.7 ms	84.1 ms
Whole tick	**97.4 ms**	**87.9 ms**

Table 6. CUDA speedup over OpenMP (2 threads) and CPU–GPU copy share of tick time, 128 instruments.

Agents	1k	4k	16k	64k	256k	1M	2M
Speedup	0.64	1.00	**1.41**	1.33	1.17	1.11	1.11
Copy share	30.2%	13.6%	6.0%	2.3%	1.0%	0.6%	0.5%

Table 7 and Fig. 8 summarise the kernel experiments. Sorting agents by strategy raises the number of active threads per warp instruction from 16.7 to 22.8 (32 means no divergence), a 27% reduction in divergence, and makes the kernel about 7% faster. Compacting the copy back to only the agents that acted

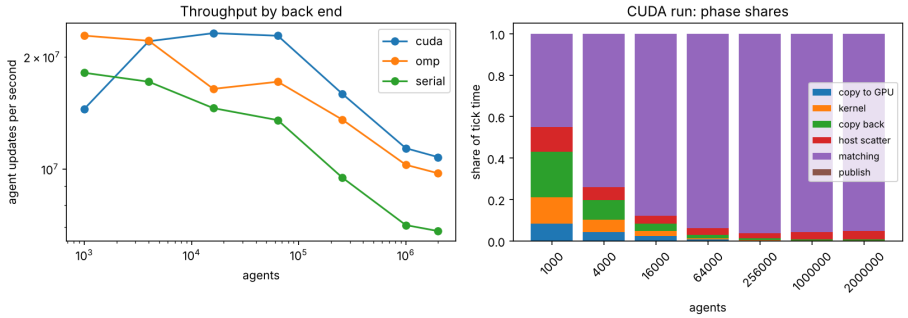


Fig. 7. CPU vs GPU crossover on the T4. Left: throughput (agent updates per second) of each back end. Right: share of tick time per phase in the CUDA runs; matching dominates beyond 16k agents.

Table 7. CUDA kernel metrics (Nsight Compute, 1M agents).

Metric	Sorted	Unsorted
Active threads per warp instr. (of 32)	**22.8**	16.7
Kernel duration	**0.138 ms**	0.149 ms
Achieved occupancy	82%	85%
Registers per thread	27	27
DRAM bandwidth	173 GB/s	168 GB/s
Copy back, full / compacted	1.90 ms / 0.46 ms	

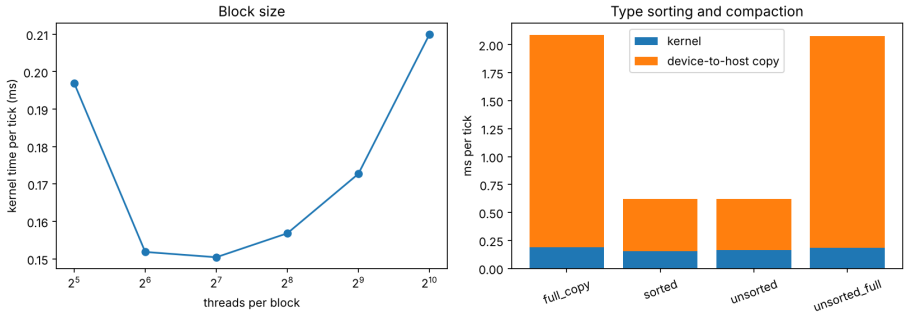


Fig. 8. CUDA kernel experiments. Left: kernel time per tick against block size. Right: kernel and device-to-host copy time with and without type sorting and compaction.

cuts it from 1.90 to 0.46 ms per tick (4.1×). Block sizes from 64 to 256 threads perform within 0.06 ms of each other; 128 is best.

Table 8. OpenMP matching speedup over serial vs number of instruments (200k agents, 16 threads).

Instruments	1	2	4	8	16	32	64	128
Speedup	0.90	0.97	1.97	2.14	2.63	3.23	3.54	**3.97**

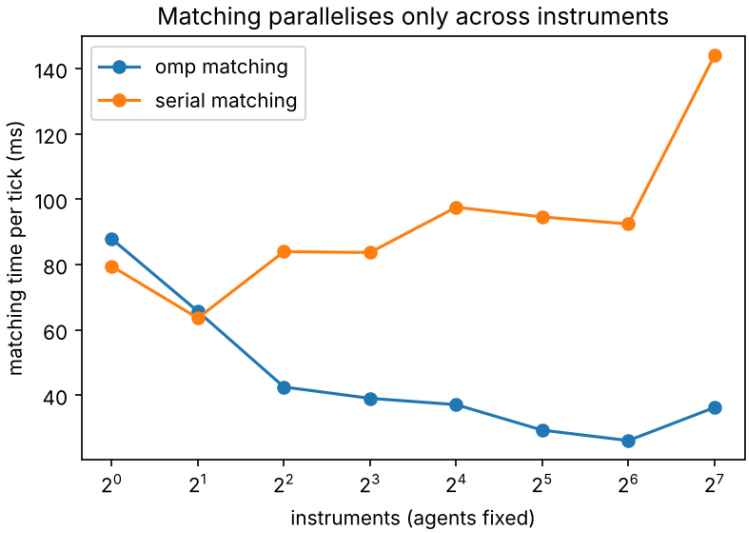


Fig. 9. Matching speedup grows only with the number of instruments.

9.3 CPU results (16-thread laptop)

Matching across instruments. Table 8 and Fig. 9 show the speedup of OpenMP matching over serial matching as instruments are added, with 16 threads. With one instrument OpenMP is slightly *slower* (0.90×): there is nothing to split, so the only effect is overhead. Speedup grows only as instruments are added, reaching 3.97× at 128.

Strong scaling. Table 9 and Fig. 10 show the whole simulation with a fixed problem. Speedup peaks at 1.99× on 8 threads and falls to 1.74× on 16. The decision phase, which needs no coordination between agents, improves only 1.75× on 8 threads (2.30 s to 1.31 s).

Weak scaling. With 50,000 agents and 8 instruments per thread (Table 10, Fig. 11), time should stay constant under ideal scaling. Instead, efficiency falls to 0.11 at 16 threads, and total throughput levels off at about 11 million agent updates per second, 1.8× the single-thread rate.

MPI. With 400,000 agents and 64 instruments split over 1, 2 and 4 ranks on the same machine (Fig. 12), communication plus waiting at the per-tick barrier takes 0%, 20% and 39% of tick time respectively. The message itself is one 64-

Table 9. Strong scaling (400k agents, 64 instruments, 300 ticks, median of 5).

Threads p	serial	1	2	4	8	16
Time (s)	17.4	17.2	14.3	9.8	8.7	10.0
Speedup S	1.00	1.01	1.21	1.78	**1.99**	1.74
Efficiency E	1.00	1.01	0.61	0.45	0.25	0.11
Karp–Flatt e	–	–	0.65	0.42	0.43	0.55

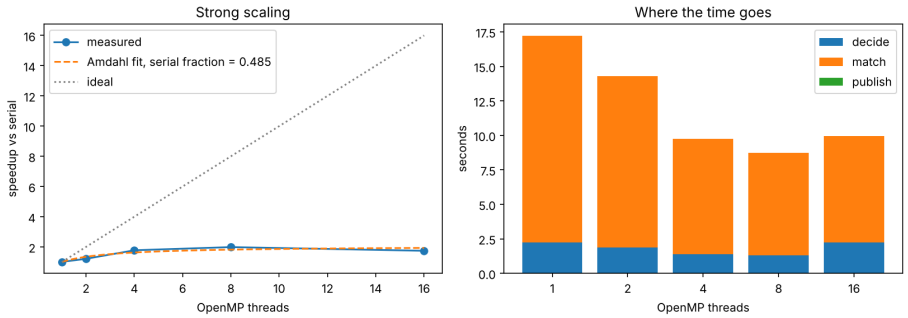


Fig. 10. Strong scaling on the laptop. Left: measured speedup, Amdahl fit (serial fraction 0.485) and ideal. Right: time per phase; matching dominates at every thread count.

Table 10. Weak scaling (50k agents and 8 instruments per thread, median of 5).

Threads	1	2	4	8	16
Time (s)	2.40	3.62	6.27	11.07	21.85
Efficiency	1.00	0.69	0.40	0.22	0.11
Updates/s (M)	6.3	8.3	9.6	10.8	11.0

bit integer; almost all of this time is ranks waiting for the slowest rank, so it measures load imbalance more than message cost. Total time barely changes, because the ranks share the same cores.

False sharing. With per-thread counters padded to separate cache lines, matching took 22.0 ms per tick; unpadded, 20.8 ms. The difference is within run-to-run noise: each thread updates the counters only a few times per instrument per tick, so cache-line contention is too rare to show. We report this as a negative result.

9.4 Model realism

Table 11 and Fig. 13 compare the simulated returns with the stylised facts of real markets [1]. The model reproduces heavy tails and volatility clustering. It fails the absence of linear autocorrelation: prices partly bounce back after a move

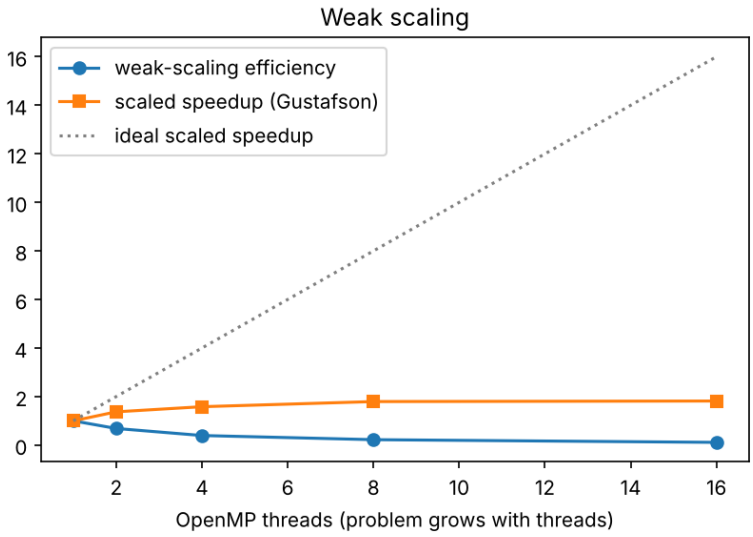


Fig. 11. Weak scaling efficiency on the laptop.

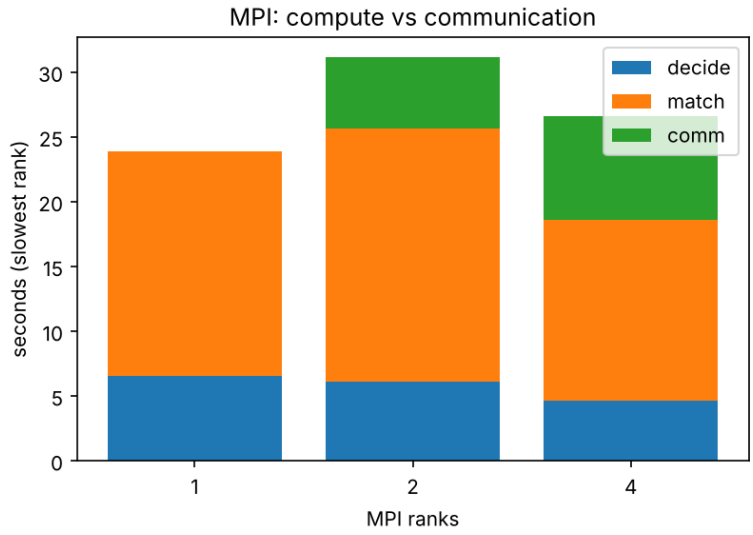


Fig. 12. MPI: share of tick time spent in communication and barrier wait.

(lag-1 autocorrelation -0.33), an effect of the mean-reverting and market-making agents.

Table 11. Stylised facts of simulated 10-tick returns (16 instruments, 20k ticks).

Property (real markets)	Measured	Pass
Heavy tails (excess kurtosis > 0)	4.26	yes
Volatility clustering ($\lvert r\rvert$ ACF lag 1 / lag 10 > 0)	0.20 / 0.09	yes
No linear autocorrelation (ACF lag 1 ≈ 0)	-0.33	no
Hill tail index (real: about 3–5)	8.0	partly

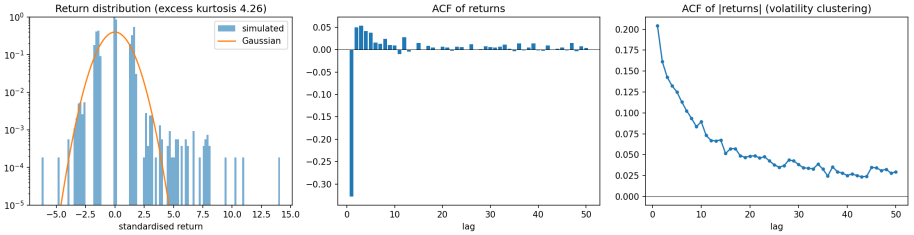


Fig. 13. Stylised facts of simulated returns: return distribution against a Gaussian, autocorrelation of returns and of absolute returns.

Table 12. Summary of speedups.

Comparison	Speedup
CUDA kernel vs 2-thread OpenMP decisions (1M agents)	85×
CUDA decision phase incl. copies and scatter	3.6×
Copy back, compacted vs full	4.1×
Kernel, type-sorted vs unsorted	1.07×
CUDA tick vs OpenMP tick (best, 16k agents)	1.41×
CUDA tick vs serial tick (32k–2M agents)	1.54–1.61×
OpenMP matching, 128 vs 1 instrument (16 threads)	3.97× vs 0.90×
Whole simulation, 8 threads (strong scaling)	1.99×
Demo run, 4-core machine: OpenMP / MPI (2 ranks)	3.40× / 3.59×

10 Performance Analysis

10.1 Speedup and efficiency

Table 12 collects the speedups. The pattern is the same everywhere: the parts that are parallel over agents speed up well (85× for the kernel), while anything that includes matching is limited to between 1.1× and 4×. Parallel efficiency is high only at low thread counts: 0.61 at 2 threads and 0.25 at 8 threads in strong scaling.

10.2 Amdahl's law: where the bound comes from

On the T4, the decision phase is 14% of the OpenMP tick at 1M agents. Even an infinitely fast GPU could then give at most $1/(1 - 0.14) \approx 1.16\times$; we mea-

sure 1.11×, close to that bound. The GPU has made decisions almost free, and matching is 92–96% of every CUDA tick (Fig. 14). This is the Amdahl bound predicted in the synopsis: matching inside one instrument is serial, and no GPU can remove it.

On the laptop, fitting Amdahl's law to the strong-scaling speedups gives an effective serial fraction of $f = 0.49$, which caps speedup at about $1/f \approx 2.1\times$. The Karp–Flatt values (0.42–0.65) are of the same size and do not fall with p, so the limit is not simple parallel overhead. The code's own serial work (publishing the index) takes about 0.001 s of 17 s, far too little to explain f. Since even the coordination-free decision phase improves only 1.75× on 8 threads, we infer that the laptop's shared resources set the limit: a power-limited mobile CPU lowers each core's clock as more cores become busy, and all cores share one memory bus, to which matching (hash lookups and pointer chasing) is especially sensitive. This interpretation is inferred from the timing pattern, not measured with hardware counters. The fitted f is therefore best read as an *effective* serial fraction: serial code plus hardware saturation.

10.3 Gustafson's law

Gustafson's law predicts that growing the problem with the machine keeps efficiency high. On the T4, growing agents at 32 fixed instruments keeps the CUDA speedup over serial at 1.54–1.61× from 32k to 2M agents (Fig. 14): the speedup does not grow, because more agents also mean more orders to match. On the laptop, weak-scaling efficiency drops to 0.11 at 16 threads and throughput levels off at 1.8× the single-thread rate. Gustafson's law assumes that the parallel part keeps scaling with more cores; that only holds while memory bandwidth and the power budget also grow.

10.4 Roofline

The decision kernel performs 3.95 integer operations per byte of DRAM traffic, below the T4's ridge point of 12.7 (4.07 Tops/s peak over 320 GB/s), so it is memory-bound (Fig. 15). It reaches 173 GB/s, 54% of peak bandwidth, at 82% occupancy with 27 registers per thread. Further kernel tuning could at most halve a phase that is already under 1% of tick time, so it would not change the end-to-end result.

10.5 CPU vs GPU threshold

Heterogeneous execution beats the multicore CPU when the GPU's saving on decisions exceeds the fixed cost of the CPU–GPU boundary (two copies, a launch and the host scatter). On the T4 this happens between 4,000 and 16,000 agents. The advantage then shrinks with size, because the matching time that the GPU cannot touch grows with the number of orders. The threshold also depends on the CPU: with more CPU threads, OpenMP decisions are faster, so the GPU needs

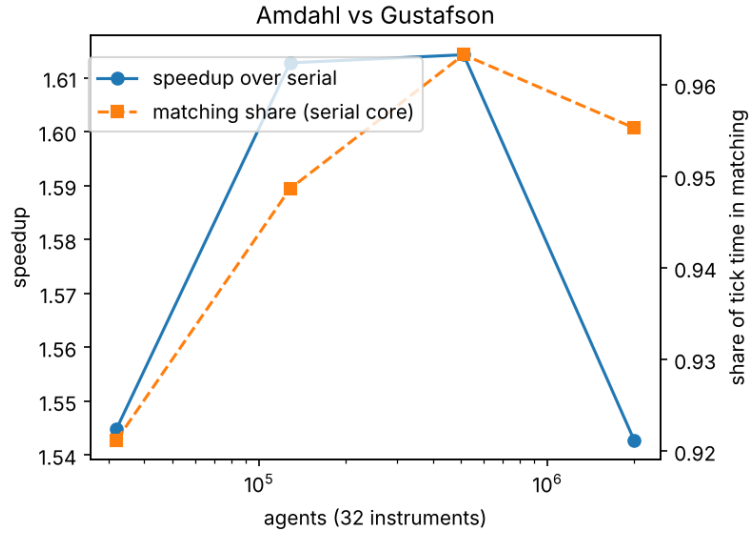


Fig. 14. Growing agents at 32 fixed instruments on the T4: CUDA speedup over serial and matching share of tick time.

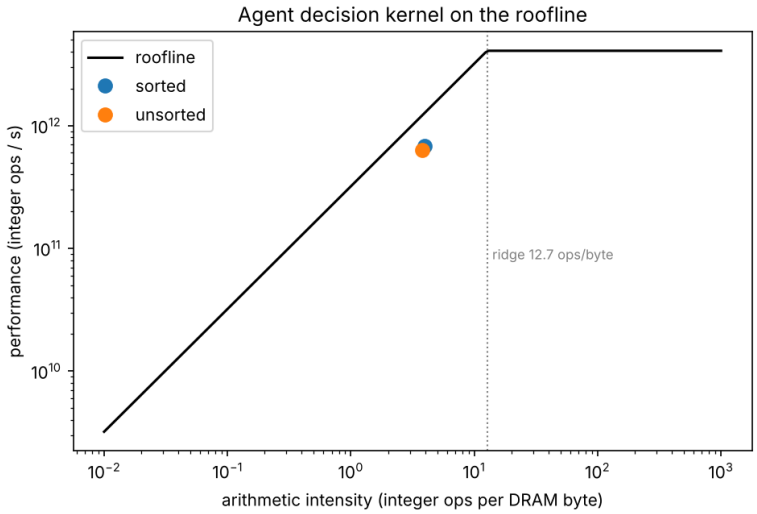


Fig. 15. Roofline placement of the decision kernel on the Tesla T4 (integer operations).

more agents to pay off. Finally, the host scatter (about 3.2 ms at 1M agents) costs 20 times the kernel; moving more of the pipeline to the GPU would only pay if matching moved too, which price–time priority prevents within an instrument.

Table 13. This work compared with existing methods.

System	What runs in parallel	Matching	Reproducibility	Scaling analysis
ABIDES / ABIDES-Gym [3,13]	nothing (serial event loop)	serial, one exchange	seed only	none
JAX-LOB [15]	independent books on GPU	serial within book, on GPU	not discussed	per-message timing
PyMarketSim [14]	nothing	serial, one asset	not discussed	single-thread throughput
FLAME GPU 2 [16]	agents on GPU (spatial models)	not applicable (bidding sub-models)	atomics non-deterministic	agent-count scaling
Repast4Py [20]	spatial partitions over MPI	not applicable	seed; cross-rank equality not addressed	strong scaling
SIMCoV-GPU [21]	spatial domains over GPUs	not applicable	statistical only (≈1%)	strong and weak scaling
This work	agents on GPU or CPU cores; instruments over threads and MPI ranks	serial within instrument, parallel across instruments	**bit-identical trade log** on all back ends	strong, weak, Amdahl fit, Karp–Flatt, roofline, crossover

10.6 Bottleneck summary

- **Matching** under price–time priority is the bottleneck on every back end: 92–96% of a CUDA tick, and it parallelises only across instruments.
- **The CPU–GPU boundary** (copies and host scatter) costs more than the kernel; compaction reduces it by 4.1×.
- **Hardware saturation** (power and memory bandwidth) limits CPU scaling to about 2× on the laptop before the code's own serial part does.
- **MPI** cost is dominated by load imbalance at the per-tick barrier, not by data volume.

11 Comparison with Existing Methods

Table 13 positions this work against the closest existing systems. Direct speed comparisons are not meaningful, because the systems simulate different models on different hardware; we compare what each one parallelises, how it handles reproducibility, and what it measures.

Same conclusion, measured. JAX-LOB states that matching inside one book is serial [15]. Our results quantify the consequence end to end: matching is 92–96% of a CUDA tick, so an 85× faster kernel yields only 1.1–1.6× for the whole simulation.

Different decomposition. Spatial ABM frameworks [16,20,21] partition by space and exchange halos. We partition by instrument, which removes all cross-rank conflicts; the only per-tick communication is one integer reduction. The cost moves to load imbalance at the barrier (20–39% of tick time with 2–4 ranks), the analogue of Repast4Py's communication-bound regime.

Stronger reproducibility. SIMCoV-GPU matches CPU and GPU results only statistically [21], and OpenRAND makes only the random numbers reproducible [22]. Counter-based random numbers, integer-only decisions, a deterministic arrival order and order-independent compaction together give a bit-identical trade log across 12 configurations, so a parallel bug shows up as a changed fingerprint.

Measured threshold. Like SIMCoV-GPU, we find the GPU pays off only above a problem size; we locate it at 4,000–16,000 agents against the 2-thread CPU and explain both the lower bound (fixed copy and launch cost) and why the advantage shrinks again (growing matching time).

Limits of this comparison. Our CPU baseline on the T4 has only 2 threads; a stronger CPU would move the crossover up. Our agents are simpler than ABIDES agents (no inventory or latency model), and the model fails the no-autocorrelation stylised fact.

12 Conclusion

We built a heterogeneous agent-based limit order book simulator with agent decisions in CUDA, matching parallelised across instruments with OpenMP, and instruments distributed with MPI. All eight objectives of the synopsis were met. Every back end produces a bit-identical trade log for a given seed, which made correctness checkable throughout. The measurements give one consistent answer: order matching under price–time priority is the scaling bottleneck. The GPU makes agent decisions about 85× faster, yet the whole tick improves by at most 1.41× over the CPU baseline, because matching takes 92–96% of the time; the measured 1.11× at 1M agents sits close to the Amdahl bound of 1.16×. Matching parallelises only across instruments (0.90× with one, 3.97× with 128). The GPU overtakes the multicore CPU between 4,000 and 16,000 agents, below which copy and launch costs dominate. On the laptop, CPU scaling levels off near 2× (effective serial fraction 0.49) because of shared power and memory limits, and MPI overhead is dominated by load imbalance at the per-tick barrier. The simulated returns show heavy tails and volatility clustering, with excess short-term mean reversion as a known limitation.

13 Future Scope

– **Faster matching.** Match independent price levels speculatively, or batch orders in periodic call auctions, which allow parallel matching within an instrument at the cost of changing market rules.

- **Moving the host scatter to the GPU.** Keep agent state and intents on the device and send per-instrument order lists directly, removing the 3.2 ms host scatter.
- **Load balancing for MPI.** Assign instruments to ranks by measured activity instead of contiguous blocks, to cut barrier waiting; and run on a real multi-node cluster to measure network cost.
- **More realistic agents.** Add inventory, cash and latency, and tune the agent mix to remove the negative lag-1 autocorrelation.
- **Stronger hardware study.** Repeat the crossover on a server CPU with many cores and on newer GPUs, and confirm the hardware-saturation explanation with performance counters.
- **Multi-GPU.** Give each MPI rank its own GPU for very large agent populations.

References

1. R. Cont, "Empirical properties of asset returns: Stylized facts and statistical issues," *Quantitative Finance*, vol. 1, no. 2, pp. 223–236, 2001.
2. T. Preis, S. Golke, W. Paul, and J. J. Schneider, "Multi-agent-based order book model of financial markets," *Europhysics Letters*, vol. 75, no. 3, pp. 510–516, 2006.
3. D. Byrd, M. Hybinette, and T. H. Balch, "ABIDES: Towards high-fidelity multi-agent market simulation," in *Proc. ACM SIGSIM Conf. Principles of Advanced Discrete Simulation (PADS)*, 2020, pp. 11–22.
4. D. K. Gode and S. Sunder, "Allocative efficiency of markets with zero-intelligence traders: Market as a partial substitute for individual rationality," *Journal of Political Economy*, vol. 101, no. 1, pp. 119–137, 1993.
5. C. Chiarella and G. Iori, "A simulation analysis of the microstructure of double auction markets," *Quantitative Finance*, vol. 2, no. 5, pp. 346–353, 2002.
6. J. D. Farmer, P. Patelli, and I. I. Zovko, "The predictive power of zero intelligence in financial markets," *Proceedings of the National Academy of Sciences*, vol. 102, no. 6, pp. 2254–2259, 2005.
7. M. Lysenko and R. M. D'Souza, "A framework for megascale agent based model simulations on graphics processing units," *Journal of Artificial Societies and Social Simulation*, vol. 11, no. 4, 2008, art. no. 10.
8. R. M. Fujimoto, "Parallel discrete event simulation," *Communications of the ACM*, vol. 33, no. 10, pp. 30–53, 1990.
9. G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.
10. J. L. Gustafson, "Reevaluating Amdahl's law," *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.
11. A. H. Karp and H. P. Flatt, "Measuring parallel processor performance," *Communications of the ACM*, vol. 33, no. 5, pp. 539–543, 1990.
12. S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
13. S. Amrouni, A. Moulin, J. Vann, S. Vyetrenko, T. Balch, and M. Veloso, "ABIDES-Gym: Gym environments for multi-agent discrete event simulation and application

to financial markets," in *Proc. 2nd ACM Int. Conf. AI in Finance (ICAIF)*, 2021, pp. 1–9.
14. C. Mascioli, A. Gu, Y. Wang, M. Chakraborty, and M. P. Wellman, "A financial market simulation environment for trading agents using deep reinforcement learning," in *Proc. 5th ACM Int. Conf. AI in Finance (ICAIF)*, 2024, pp. 117–125.
15. S. Frey, K. Li, P. Nagy, S. Sapora, C. Lu, S. Zohren, J. Foerster, and A. Calinescu, "JAX-LOB: A GPU-accelerated limit order book simulator to unlock large scale reinforcement learning for trading," in *Proc. 4th ACM Int. Conf. AI in Finance (ICAIF)*, 2023, pp. 583–591.
16. P. Richmond, R. Chisholm, P. Heywood, M. K. Chimeh, and M. Leach, "FLAME GPU 2: A framework for flexible and performant agent based simulation on GPUs," *Software: Practice and Experience*, vol. 53, no. 8, pp. 1659–1680, 2023.
17. G. Datseris, A. R. Vahdati, and T. C. DuBois, "Agents.jl: A performant and feature-full agent-based modeling software of minimal code complexity," *SIMULATION*, vol. 100, no. 10, pp. 1019–1031, 2024.
18. I. Jericevich, D. Sing, and T. Gebbie, "CoinTossX: An open-source low-latency high-throughput matching engine," *SoftwareX*, vol. 19, 2022, art. no. 101136.
19. S. Yoo, H. Kim, J. Kim, S. Park, J.-Y. Kim, and J. Oh, "LightTrader: A standalone high-frequency trading system with deep learning inference accelerators and proactive scheduler," in *Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA)*, 2023, pp. 1017–1030.
20. N. Collier and J. Ozik, "Distributed agent-based simulation with Repast4Py," in *Proc. Winter Simulation Conf. (WSC)*, 2022, pp. 192–206.
21. K. G. Leyba, S. Hofmeyr, J. Cannon, M. Moses, and S. Forrest, "SIMCoV-GPU: Accelerating an agent-based model for exascale," in *Proc. 33rd Int. Symp. High-Performance Parallel and Distributed Computing (HPDC)*, 2024, pp. 322–333.
22. S. S. Khan, B. Palmer, C. Edelmaier, and H. M. Aktulga, "OpenRAND: A performance portable, reproducible random number generation library for parallel computations," *SoftwareX*, vol. 27, 2024, art. no. 101773.
23. J. K. Salmon, M. A. Moraes, R. O. Dror, and D. E. Shaw, "Parallel random numbers: As easy as 1, 2, 3," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*, 2011, pp. 1–12.